Projet 4 : Validation croisée et Random Forests

Margot Laignel-Arthur Hauguel-Loïc Mémeteau

Introduction

Les Random Forests sont des méthodes d'apprentissage supervisé ensemblistes qui agrègent de nombreux arbres de décision construits de manière stochastique. Ce projet vise à estimer, par validation croisée à 10 folds, les performances de classification de ce modèle pour plusieurs valeurs de `ntree`, puis à comparer ces résultats à ceux de l'algorithme des K plus proches voisins (KNN).
Les métriques étudiées sont l'`accuracy`, la `sensitivity` et la `specificity`, avec la classe `1` définie comme classe positive et la classe `0` comme classe négative.

Description des Données

Le fichier `bacteria-dataset.Rdata` contient : - une matrice `X` de taille 2403 x 240 (signaux spectraux), - un vecteur `Y` de longueur 2403 (étiquettes de classe 0/1).

```r
dim(X)
```

```
## [1] 2403  240
```

```r
length(Y)
```

```
## [1] 2403
```

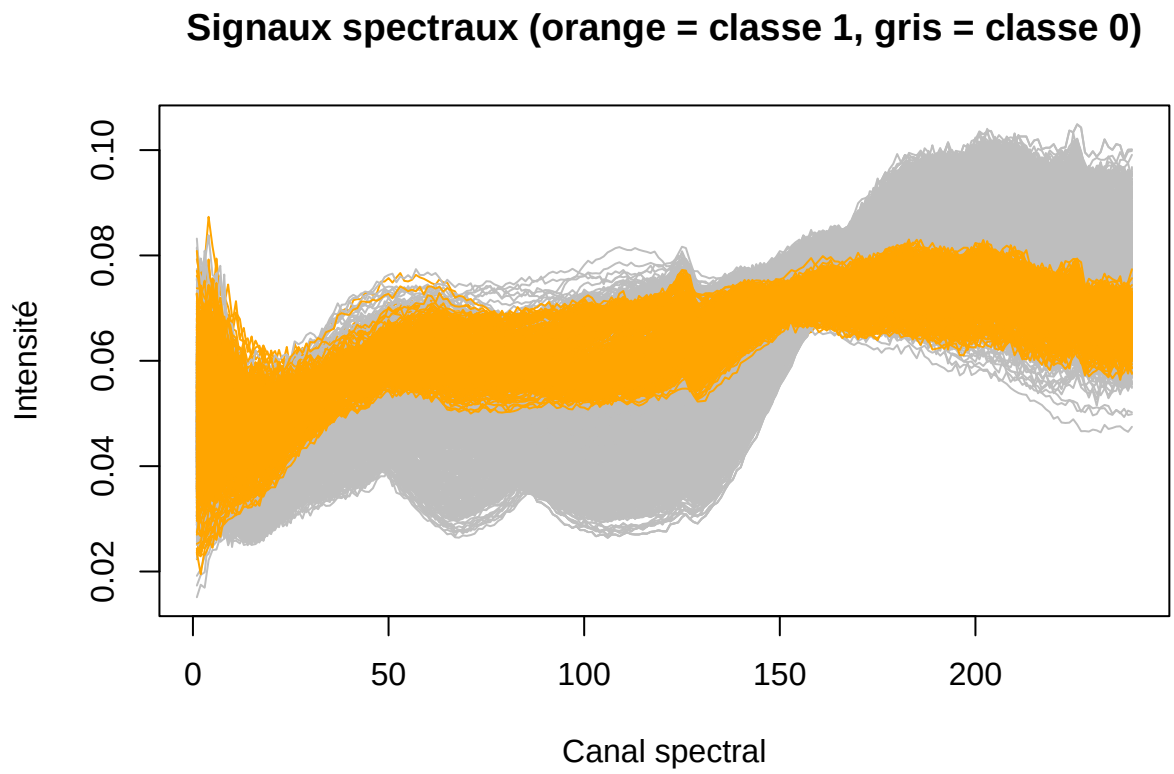
```r
table(Y)
```

```
## Y
##    0    1
## 1839  564
```

```r
prop.table(table(Y))
```

```
## Y
##         0         1
## 0.7652934 0.2347066
```

```r
matplot(
  t(X),
  type = "l",
  lty = 1,
  col = ifelse(Y == 1, "orange", "grey"),
  main = "Signaux spectraux (orange = classe 1, gris = classe 0)",
  xlab = "Canal spectral",
  ylab = "Intensité"
)
```



Dans toute la suite, on considère explicitement : - classe positive : `1`, - classe négative : `0`.

Méthodologie

Les Random Forests contrôlent leur complexité principalement via : - `ntree` : nombre d'arbres agrégés, - `mtry` : nombre de variables candidates testées à chaque split.

Dans le package `randomForest`, pour une tâche de classification avec `p` variables, la valeur par défaut est `mtry = floor(sqrt(p))`. Ici `p = 240`, donc `mtry` par défaut vaut :

```
floor(sqrt(ncol(X)))
```

```
## [1] 15
```

Un `ntree` plus élevé peut stabiliser la variance du modèle mais augmente le coût de calcul. Le paramètre `mtry` agit sur le niveau d'aléa des arbres, donc sur le compromis biais/variance (et indirectement sur la complexité effective du modèle). L'évaluation est faite par validation croisée stratifiée à 10 folds.

```
parametres_ntree <- c(25, 50, 100, 250, 500, 1000)
parametres_k <- 1:9
kfolds <- 10
```

```
accuracy <- function(true_positive, true_negative, false_positive, false_negative) {
  total <- true_positive + true_negative + false_positive + false_negative
  if (total == 0) return(NA_real_)
  (true_positive + true_negative) / total
}
```

```r
sensitivity <- function(true_positive, false_negative) {
  denom <- true_positive + false_negative
  if (denom == 0) return(NA_real_)
  true_positive / denom
}

specificity <- function(true_negative, false_positive) {
  denom <- true_negative + false_positive
  if (denom == 0) return(NA_real_)
  true_negative / denom
}

results <- function(y_true, y_pred) {
  y_true <- as.character(y_true)
  y_pred <- as.character(y_pred)

  true_positive <- sum(y_true == "1" & y_pred == "1")
  false_positive <- sum(y_true == "0" & y_pred == "1")
  true_negative <- sum(y_true == "0" & y_pred == "0")
  false_negative <- sum(y_true == "1" & y_pred == "0")

  c(
    accuracy = accuracy(true_positive, true_negative, false_positive, false_negative),
    sensitivity = sensitivity(true_positive, false_negative),
    specificity = specificity(true_negative, false_positive)
  )
}
```

```r
# Découpage stratifié en 10 folds
idx_pos <- sample(which(as.character(Y) == "1"))
idx_neg <- sample(which(as.character(Y) == "0"))

fold_id <- integer(length(Y))
fold_id[idx_pos] <- rep(seq_len(kfolds), length.out = length(idx_pos))
fold_id[idx_neg] <- rep(seq_len(kfolds), length.out = length(idx_neg))

folds <- lapply(seq_len(kfolds), function(f) which(fold_id == f))

# Contrôle de taille des folds
sapply(folds, length)
```

```
##  [1] 241 241 241 241 240 240 240 240 240 239
```

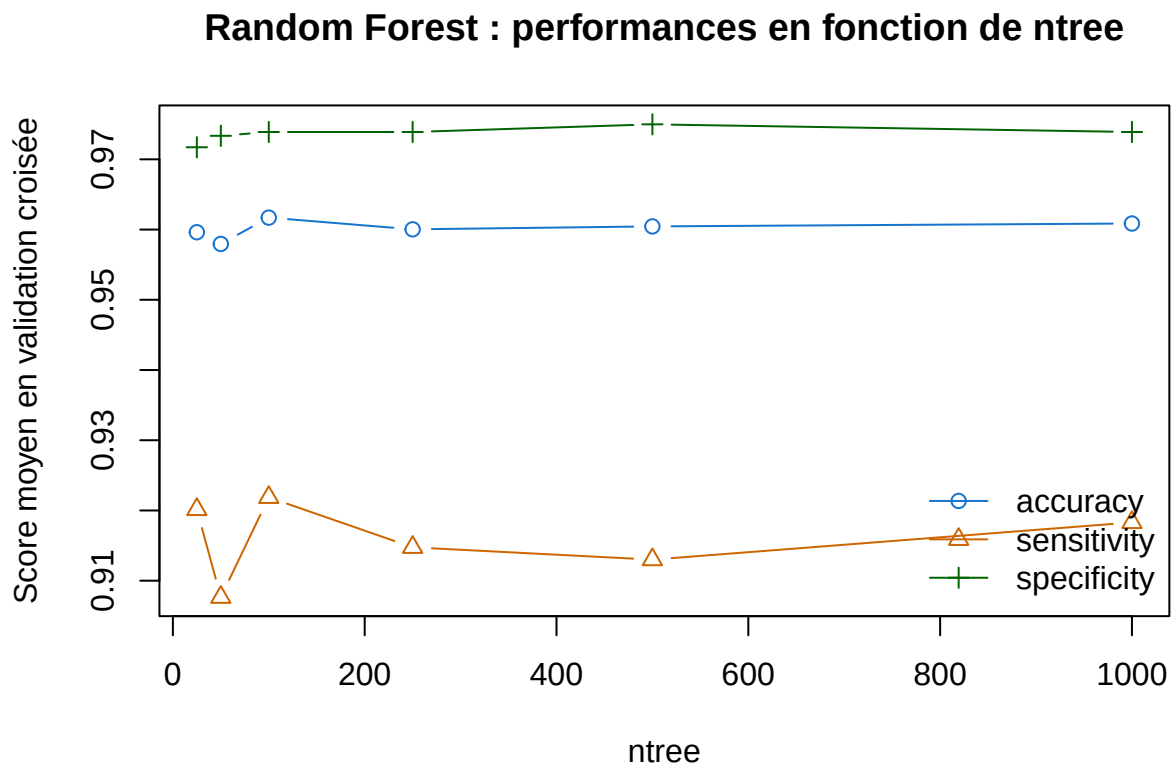
Résultats Random Forest

```r
rf_perf <- matrix(0, nrow = length(parametres_ntree), ncol = 3)
colnames(rf_perf) <- c("accuracy", "sensitivity", "specificity")
rownames(rf_perf) <- parametres_ntree

for (i in seq_along(parametres_ntree)) {
  n <- parametres_ntree[i]

  scores_folds <- sapply(seq_len(kfolds), function(f) {
```

```r
    test_idx <- folds[[f]]
    train_idx <- setdiff(seq_len(nrow(X)), test_idx)

    mod <- randomForest(
      x = X[train_idx, , drop = FALSE],
      y = as.factor(Y[train_idx]),
      ntree = n
    )
    preds <- predict(mod, X[test_idx, , drop = FALSE])
    results(Y[test_idx], preds)
  })

  rf_perf[i, ] <- rowMeans(scores_folds)
}

rf_perf
```

```
##       accuracy sensitivity specificity
## 25   0.9596212   0.9201441   0.9717183
## 50   0.9579545   0.9076441   0.9733517
## 100  0.9617011   0.9218672   0.9738952
## 250  0.9600327   0.9147556   0.9738952
## 500  0.9604493   0.9130013   0.9749822
## 1000 0.9608625   0.9182957   0.9738952
```

```r
matplot(
  x = parametres_ntree,
  y = rf_perf,
  type = "b",
  pch = 1:3,
  lty = 1,
  col = c("dodgerblue3", "darkorange3", "darkgreen"),
  xlab = "ntree",
  ylab = "Score moyen en validation croisée",
  main = "Random Forest : performances en fonction de ntree"
)
legend(
  "bottomright",
  legend = c("accuracy", "sensitivity", "specificity"),
  col = c("dodgerblue3", "darkorange3", "darkgreen"),
  lty = 1,
  pch = 1:3,
  bty = "n"
)
```




```
max_acc_rf <- max(rf_perf[, "accuracy"])
best_ntree <- min(as.numeric(rownames(rf_perf)[rf_perf[, "accuracy"] == max_acc_rf]))

cat(
  sprintf(
    "Le meilleur ntree (critère accuracy) est %d, avec accuracy = %.4f, sensitivity = %.4f, specificity
    best_ntree,
    rf_perf[as.character(best_ntree), "accuracy"],
    rf_perf[as.character(best_ntree), "sensitivity"],
    rf_perf[as.character(best_ntree), "specificity"]
  )
)
```

```
## Le meilleur ntree (critère accuracy) est 100, avec accuracy = 0.9617, sensitivity = 0.9219, specific
```

La valeur retenue pour `ntree` est celle qui maximise l'`accuracy` moyenne en validation croisée (avec choix de la plus petite valeur en cas d'égalité), afin de limiter la complexité de calcul à performance équivalente.

Résultats KNN

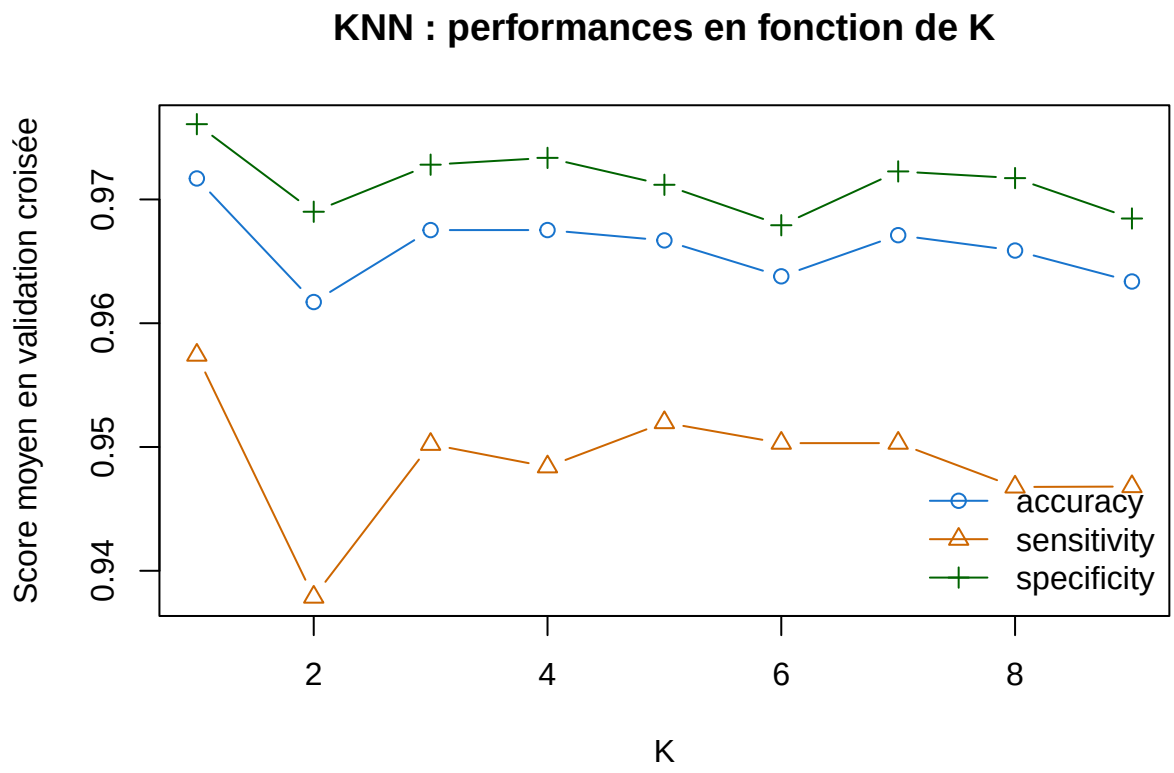
```
knn_perf <- matrix(0, nrow = length(parametres_k), ncol = 3)
colnames(knn_perf) <- c("accuracy", "sensitivity", "specificity")
rownames(knn_perf) <- parametres_k

for (i in seq_along(parametres_k)) {
  k <- parametres_k[i]
```

```r
  scores_folds <- sapply(seq_len(kfolds), function(f) {
    test_idx <- folds[[f]]
    train_idx <- setdiff(seq_len(nrow(X)), test_idx)

    # Standardisation par fold (sans fuite d'information)
    x_train <- scale(X[train_idx, , drop = FALSE])
    center_train <- attr(x_train, "scaled:center")
    scale_train <- attr(x_train, "scaled:scale")
    scale_train[scale_train == 0] <- 1
    x_test <- scale(X[test_idx, , drop = FALSE], center = center_train, scale = scale_train)

    preds <- class::knn(
      train = x_train,
      test = x_test,
      cl = as.factor(Y[train_idx]),
      k = k
    )
    results(Y[test_idx], preds)
  })

  knn_perf[i, ] <- rowMeans(scores_folds)
}

knn_perf
```

```
##    accuracy sensitivity specificity
## 1 0.9716959   0.9574248   0.9760810
## 2 0.9617115   0.9378759   0.9690069
## 3 0.9675240   0.9502193   0.9728083
## 4 0.9675258   0.9484023   0.9733577
## 5 0.9666907   0.9519737   0.9711838
## 6 0.9637878   0.9503133   0.9679170
## 7 0.9671125   0.9503133   0.9722648
## 8 0.9658712   0.9467732   0.9717243
## 9 0.9633712   0.9468045   0.9684604
```

```r
matplot(
  x = parametres_k,
  y = knn_perf,
  type = "b",
  pch = 1:3,
  lty = 1,
  col = c("dodgerblue3", "darkorange3", "darkgreen"),
  xlab = "K",
  ylab = "Score moyen en validation croisée",
  main = "KNN : performances en fonction de K"
)
legend(
  "bottomright",
  legend = c("accuracy", "sensitivity", "specificity"),
  col = c("dodgerblue3", "darkorange3", "darkgreen"),
  lty = 1,
  pch = 1:3,
  bty = "n"
```

```
)
```




```
max_acc_knn <- max(knn_perf[, "accuracy"])
best_k <- min(as.numeric(rownames(knn_perf)[knn_perf[, "accuracy"] == max_acc_knn]))

cat(
  sprintf(
    "Le meilleur K (critère accuracy) est %d, avec accuracy = %.4f, sensitivity = %.4f, specificity = %
    best_k,
    knn_perf[as.character(best_k), "accuracy"],
    knn_perf[as.character(best_k), "sensitivity"],
    knn_perf[as.character(best_k), "specificity"]
  )
)
```

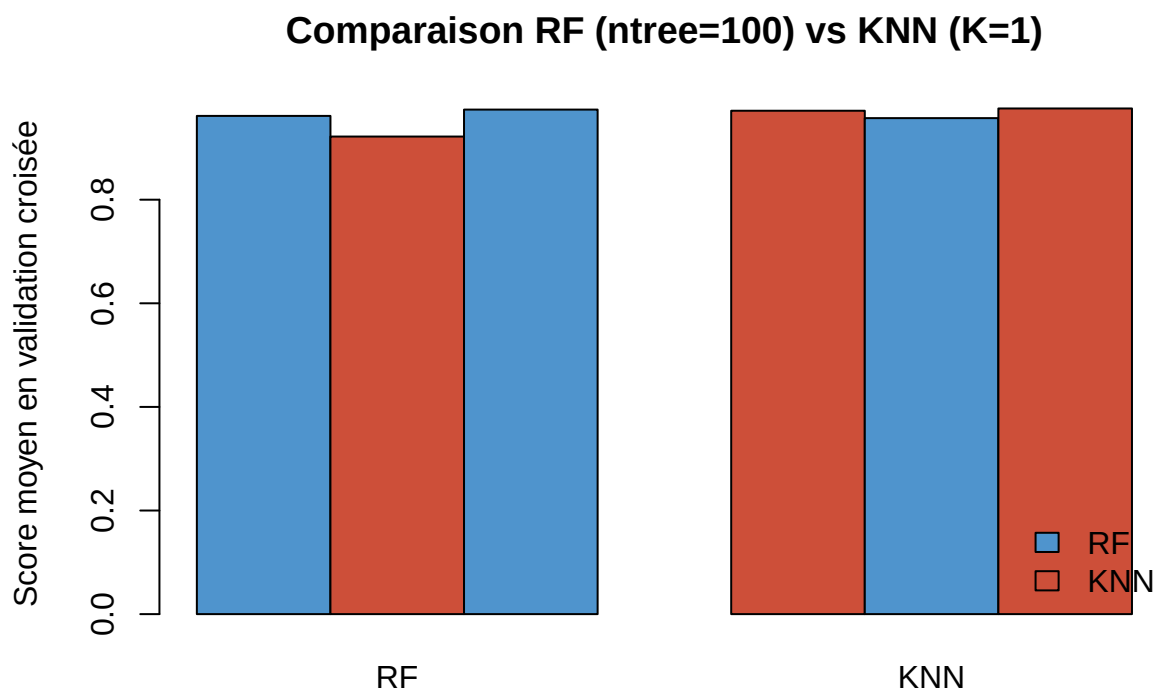
```
## Le meilleur K (critère accuracy) est 1, avec accuracy = 0.9717, sensitivity = 0.9574, specificity = 
```

Comparaison RF vs KNN

```
comparaison <- rbind(
  RF = rf_perf[as.character(best_ntree), ],
  KNN = knn_perf[as.character(best_k), ]
)

comparaison
```

```
##      accuracy sensitivity specificity
## RF  0.9617011   0.9218672   0.9738952
## KNN 0.9716959   0.9574248   0.9760810
```

```r
barplot(
  t(comparaison),
  beside = TRUE,
  col = c("steelblue3", "tomato3"),
  ylab = "Score moyen en validation croisée",
  main = paste0("Comparaison RF (ntree=", best_ntree, ") vs KNN (K=", best_k, ")")
)
legend(
  "bottomright",
  legend = c("RF", "KNN"),
  fill = c("steelblue3", "tomato3"),
  bty = "n"
)
```




```r
delta <- comparaison["RF", ] - comparaison["KNN", ]

cat(
  sprintf(
    paste0(
      "Différences RF - KNN : accuracy = %.4f, sensitivity = %.4f, specificity = %.4f. ",
      "Ces écarts permettent d'identifier si RF domine globalement ou si le gain dépend de la métrique :
    ),
    delta["accuracy"],
```

```
    delta["sensitivity"],
    delta["specificity"]
  )
)
```

```
## Différences RF - KNN : accuracy = -0.0100, sensitivity = -0.0356, specificity = -0.0022. Ces écarts p
```

Analyse détaillée de mtry

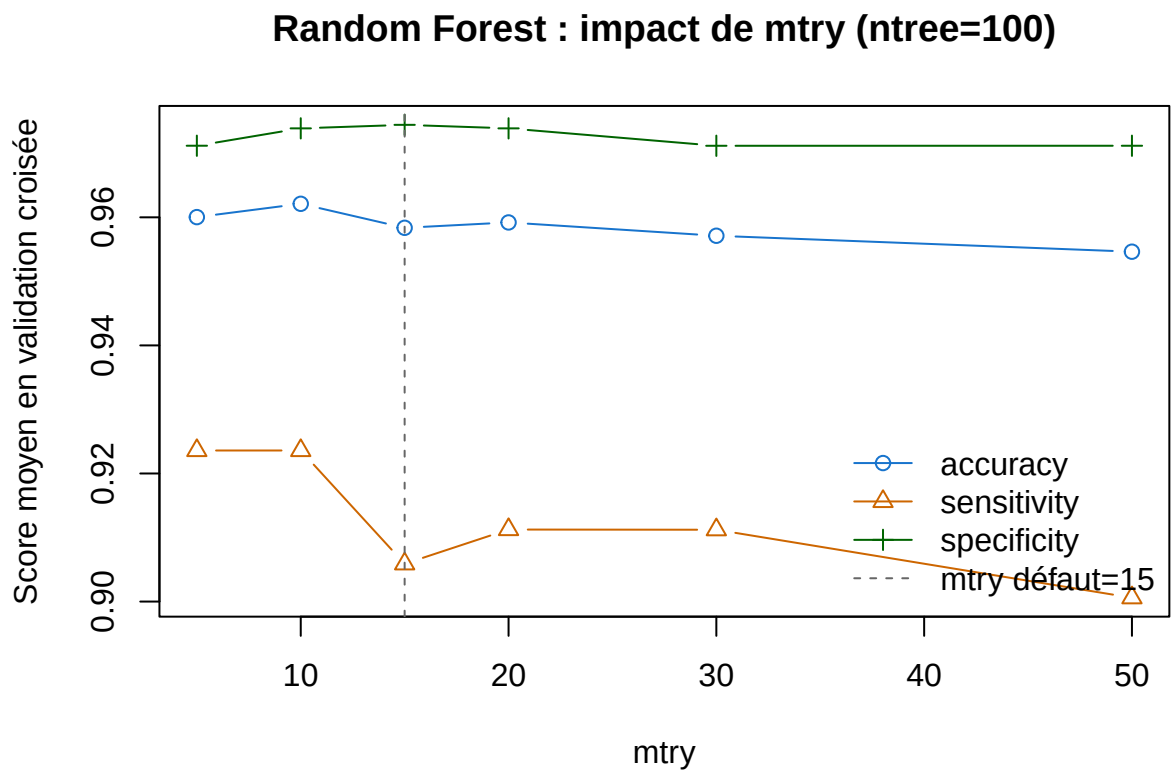
La consigne indique que l'étude de `mtry` est optionnelle. Ici elle est approfondie en fixant `ntree` à la meilleure valeur trouvée précédemment.

```
default_mtry <- floor(sqrt(ncol(X)))
mtry_values <- sort(unique(c(5, 10, default_mtry, 20, 30, 50)))
mtry_values <- mtry_values[mtry_values >= 1 & mtry_values <= ncol(X)]

mtry_perf <- matrix(0, nrow = length(mtry_values), ncol = 3)
colnames(mtry_perf) <- c("accuracy", "sensitivity", "specificity")
rownames(mtry_perf) <- mtry_values

for (i in seq_along(mtry_values)) {
  m <- mtry_values[i]

  scores_folds <- sapply(seq_len(kfolds), function(f) {
    test_idx <- folds[[f]]
    train_idx <- setdiff(seq_len(nrow(X)), test_idx)

    mod <- randomForest(
      x = X[train_idx, , drop = FALSE],
      y = as.factor(Y[train_idx]),
      ntree = best_ntree,
      mtry = m
    )
    preds <- predict(mod, X[test_idx, , drop = FALSE])
    results(Y[test_idx], preds)
  })

  mtry_perf[i, ] <- rowMeans(scores_folds)
}

mtry_perf
```

```
##     accuracy sensitivity specificity
## 5  0.9600361   0.9235902   0.9711808
## 10 0.9621125   0.9235902   0.9738952
## 15 0.9583660   0.9058897   0.9744417
## 20 0.9592028   0.9112469   0.9738952
## 30 0.9571229   0.9112155   0.9711749
## 50 0.9546333   0.9005952   0.9711778
```

```
matplot(
  x = mtry_values,
  y = mtry_perf,
  type = "b",
```

```r
  pch = 1:3,
  lty = 1,
  col = c("dodgerblue3", "darkorange3", "darkgreen"),
  xlab = "mtry",
  ylab = "Score moyen en validation croisée",
  main = paste0("Random Forest : impact de mtry (ntree=", best_ntree, ")")
)
abline(v = default_mtry, lty = 2, col = "grey40")
legend(
  "bottomright",
  legend = c("accuracy", "sensitivity", "specificity", paste0("mtry défaut=", default_mtry)),
  col = c("dodgerblue3", "darkorange3", "darkgreen", "grey40"),
  lty = c(1, 1, 1, 2),
  pch = c(1, 2, 3, NA),
  bty = "n"
)
```




```r
max_acc_mtry <- max(mtry_perf[, "accuracy"])
best_mtry <- min(as.numeric(rownames(mtry_perf)[mtry_perf[, "accuracy"] == max_acc_mtry]))

cat(
  sprintf(
    "Le meilleur mtry testé est %d (mtry par défaut = %d), avec accuracy = %.4f.",
    best_mtry,
    default_mtry,
    mtry_perf[as.character(best_mtry), "accuracy"]
```

```
  )
)
```

```
## Le meilleur mtry testé est 10 (mtry par défaut = 15), avec accuracy = 0.9621.
```

Contrôles de cohérence

```
check_metrics <- function(m) {
  all(!is.na(m)) && all(m >= 0) && all(m <= 1)
}

c(
  rf_ok = check_metrics(rf_perf),
  knn_ok = check_metrics(knn_perf),
  mtry_ok = check_metrics(mtry_perf)
)
```

```
##   rf_ok  knn_ok mtry_ok
##    TRUE    TRUE    TRUE
```

Conclusion

Ce travail met en place une validation croisée stratifiée à 10 folds pour comparer Random Forest et KNN sur les métriques `accuracy`, `sensitivity` et `specificity`. La comparaison permet de sélectionner un `ntree` optimal pour RF et un `K` optimal pour KNN selon l'`accuracy` moyenne, puis d'évaluer l'effet du paramètre `mtry` autour de sa valeur par défaut (`15`).
Globalement, la méthode retenue est celle offrant le meilleur compromis entre performance moyenne et complexité de réglage, en s'appuyant sur des métriques calculées manuellement à partir de la matrice de confusion.